

编译技术

代码优化设计与实现

林子淇

目录

一、 优化目标与策略	1
二、 Pass 管理机制	1
三、 预处理优化	2
3.1 函数内联	2
3.2 全局变量局部化	3
四、 SSA 构造与标量优化	4
4.1 SSA 构造	4
4.2 CFG 清理	5
4.3 常量传播	5
五、 循环与全局优化	6
5.1 循环变换	6
5.2 全局值编号	7
5.3 代码移动	7
5.4 强度削减	8
六、 后端代码生成	8
6.1 Phi 消除	8
6.2 寄存器分配	9
6.3 除法优化	10
七、 总结	10

一、优化目标与策略

本项目的优化目标直接受评测指标 `FinalCycle` 约束。以 MIPS 目标为例，测试框架根据指令类型赋予不同权重，其中除法、乘法与访存指令开销显著：

$$\text{FinalCycle} = 15c \cdot \text{DIV} + 5c \cdot \text{MULT} + 2c \cdot (\text{JUMP} + \text{BRANCH}) + 3c \cdot \text{MEM} + 1c \cdot \text{OTHER}$$

该权重分配确立了两个核心优化方向：一是通过强度削减与循环变换减少高权重的 `DIV / MULT` 指令，或将其替换为代价更低的位运算序列；二是利用中端优化消除冗余访存（如全局变量局部化、标量替换），减轻后端寄存器分配压力，从而降低 `MEM` 类指令的动态频次。

在流水线设计上，`OptimizeStage` 作为独立阶段插入在 IR 生成与汇编生成之间。当 `CompilerConfig.OPTIMIZE` 开启时，编译器在语义分析与 IR 生成后执行优化 Pass，再将优化后的 IR 传递给后端。

```
// src/top/tsxb/compiler/driver/Pipeline.java
stages.put("llvm", new IrGenStage());
if (CompilerConfig.OPTIMIZE) {
    stages.put("optimize", new OptimizeStage()); // 插入优化阶段
}
stages.put("mips", new MipsGenStage());
```

代码 1 `OptimizeStage` 阶段在流水线中的位置

通过分层设计，中端和后端可以独立进行算法迭代：中端专注 IR 层面的等价变换与控制流简化（如常量传播、死代码消除），旨在降低计算复杂度；后端专注指令选择与资源分配（如寄存器分配、窥孔优化），旨在将 IR 高效映射到硬件指令。

二、Pass 管理机制

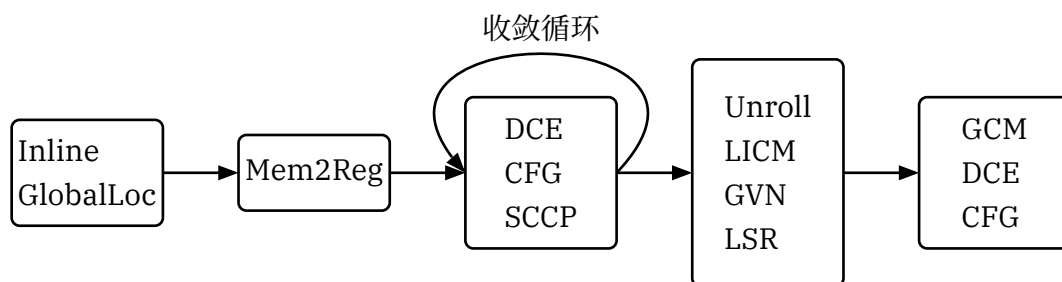


图 1 Pass 执行依赖关系示意图

Pass 管理器采用“反复执行直至收敛”的策略。`PassManager` 维护一个有序的 Pass 列表，通过 `changed` 标志判断本轮迭代是否对 IR 进行了修改。为了防止因非单调变换导致的死循环，管理器设置了最大迭代次数 `MAX_ITERATIONS`。同时，`ErrorReporter` 在每个 Pass 执行前检查错误状态，确保在发生不可恢复错误时及时终止。

默认流水线遵循“构造-变换-清理”的拓扑顺序。首先通过内联与全局变量局部化暴露优化机会，随后进入 SSA 构造。在 SSA 形式下，交替执行死代码消除（DCE）、控制流简化

(SimplifyCFG) 与常量传播 (SCCP), 逐步收敛 IR 状态。循环优化 (如 LICM、GVN) 安排在结构稳定之后, 最后通过全局代码移动 (GCM) 进行指令调度。

```
while (changed && iterations < MAX_ITERATIONS) {
    changed = false;
    for (Pass pass : passes) {
        if (errorReporter.hasErrors()) return;
        // 只要任意 Pass 改变了 IR, changed 即为 true, 驱动下一轮迭代
        changed |= pass.run(module);
    }
    iterations++;
}
```

代码 2 PassManager 迭代执行逻辑

三、预处理优化

预处理阶段旨在调整 IR 结构, 为后续 SSA 构造与标量优化铺路。通过消除过程边界与全局状态依赖, 使后续 Pass 能在更局部的范围内进行分析。

3.1 函数内联

FunctionInliningPass 采用“指令规模阈值 + 递归特判”的策略。对于普通函数, 仅当指令数小于 MAX_INLINE_SIZE 时内联; 对于递归函数, 为了控制代码膨胀, 要求实参必须全为常量且指令数限制更宽松。这种策略在消除调用开销与控制代码体积之间取得了平衡。

```
BasicBlock afterBlock = new BasicBlock(bb.getName() + ".inline.after", caller);
// ... 将调用点之后的指令移动到 afterBlock ...
// 建立参数映射: 形参 → 实参
for (int i = 0; i < call.getNumOperands() - 1; i++) {
    valueMap.put(callee.getArguments().get(i), call.getOperand(i + 1));
}
// 克隆被调函数的基本块与指令
for (BasicBlock calleeBB : rpo) {
    BasicBlock newBB = new BasicBlock(calleeBB.getName(), caller);
    blockMap.put(calleeBB, newBB);
    for (Instruction inst : calleeBB.getInstructions()) {
        // 特殊处理: 将 alloca 提升到调用者入口块, 避免栈帧混乱
        if (inst instanceof AllocaInst) {
            Instruction newAlloca = inst.clone();
            callerEntry.addFirst(newAlloca);
            valueMap.put(inst, newAlloca);
        } else {
            // ... 克隆普通指令并更新操作数引用 ...
        }
    }
}
```

代码 3 函数内联核心逻辑

内联过程涉及控制流图的克隆与重接。调用点被拆分为调用前与调用后两个基本块，被调函数的 CFG 被完整复制并插入其中。参数传递通过值映射机制完成，返回值则通过新建的 Phi 指令汇聚。

3.2 全局变量局部化

`GlobalLocalizationPass` 将非 `const` 全局变量转化为局部栈槽。该 Pass 统计全局变量的使用情况，若变量仅在单函数内使用，或虽跨函数但在循环内高频访问，则将其局部化。局部化后，原本的全局 `load / store` 转化为对 `alloca` 的操作，并在函数入口与出口（或调用点）插入必要的同步指令，从而将访存操作暴露给后续的 `Mem2Reg` 进行提升。

```
// 在调用点插入同步指令
if (needsStore(gv, call)) {
    // 调用前：将局部值写回全局
    new StoreInst(localAlloca, gv, call);
}
// ... 执行调用 ...
if (needsReload(gv, call)) {
    // 调用后：从全局读回局部
    new LoadInst(gv, call.getNext());
    new StoreInst(loadResult, localAlloca, call.getNext().getNext());
}
```

代码 4 全局变量局部化同步指令插入

四、SSA 构造与标量优化

中端优化的核心是将隐式的内存依赖转化为显式的 SSA 值依赖。在此基础上，通过一系列标量优化 Pass 消除冗余计算与不可达路径。

4.1 SSA 构造

`Mem2RegPass` 负责将局部 `alloca` 提升为 SSA 值。它首先计算支配树与支配边界，确定需要在哪些基本块插入 Phi 指令；随后在支配树上进行重命名，将对 `alloca` 的 `load/store` 替换为 SSA 值与 Phi 节点。

Phi 插入采用基于工作队列的迭代算法。一旦某个块插入了 Phi，它就成为新的定义点，可能触发其支配边界上的进一步插入。

```
// 重命名过程：利用栈维护当前活跃的 SSA 值
private void rename(BasicBlock bb) {
    // 1. 处理当前块的 Phi 定义
    for (PhiInst phi : phiMap.get(bb)) {
        stack.push(phi);
    }
    // 2. 处理块内指令：Load 替换为栈顶值，Store 压入新值
    for (Instruction inst : bb.getInstructions()) {
        if (inst instanceof LoadInst load && load.getPointer() == alloca) {
            load.replaceAllUsesWith(stack.peek());
            inst.remove();
        } else if (inst instanceof StoreInst store && store.getPointer() ==
alloca) {
            stack.push(store.getValue());
            inst.remove();
        }
    }
    // 3. 填充后继块 Phi 的入边
    for (BasicBlock succ : bb.getSuccessors()) {
        PhiInst phi = phiMap.get(succ).get(alloca);
        if (phi != null) phi.addIncoming(stack.peek(), bb);
    }
    // 4. 递归处理支配树子节点
    for (BasicBlock child : domTree.getChildren(bb)) rename(child);
    // 5. 回溯：恢复栈状态
    // ... pop stack ...
}
```

代码 5 `Mem2Reg` 重命名过程

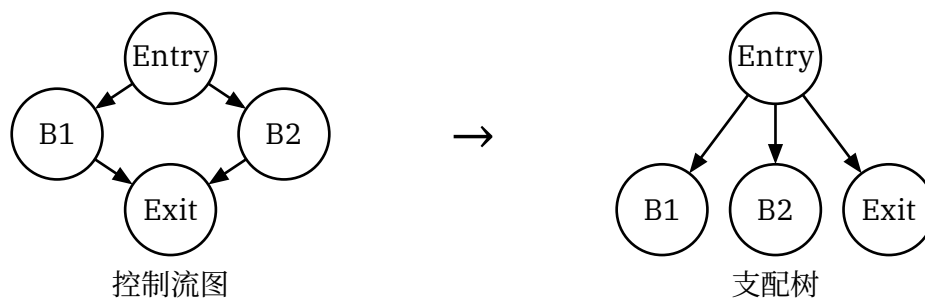


图 2 支配树与支配边界构建示例

4.2 CFG 清理

`DeadCodeEliminationPass` (DCE) 与 `SimplifyCfgPass` 协同工作，负责清理 IR 中的冗余结构。DCE 移除无副作用且结果未被使用的指令，并清理死存储；`SimplifyCFG` 则合并线性基本块、移除不可达块与空跳转块。这两个 Pass 在 SSA 构造及其他变换后反复执行，保持控制流图的紧凑性。

4.3 常量传播

常量传播基于 SCCP (Sparse Conditional Constant Propagation) 算法，在格 (Lattice) 上进行值推导。格值分为 `TOP` (未知)、`CONST` (常量) 与 `BOTTOM` (非常量)。

```

// 指令求值逻辑：利用格值进行运算
private void visitBinary(BinaryInst inst) {
    LatticeValue v1 = getValueState(inst.getOperand(0));
    LatticeValue v2 = getValueState(inst.getOperand(1));

    if (v1.isConstant() && v2.isConstant()) {
        // 双方均为常量：直接折叠
        int res = fold(inst.getOpCode(), v1.getInt(), v2.getInt());
        updateValueState(inst, LatticeValue.constant(res));
    } else if (v1.isBottom() || v2.isBottom()) {
        // 任意一方为 Bottom：结果为 Bottom
        updateValueState(inst, LatticeValue.bottom());
    }
    // 否则保持 Top，等待后续更新
}
  
```

代码 6 SCCP 指令求值逻辑

SCCP 同时维护控制流的可达性状态。仅当控制流边被标记为可执行时，对应的 Phi 入边才参与计算。这使得算法能同时移除死代码分支与折叠常量计算。`IpsccpPass` 将此逻辑扩展到跨过程分析，利用调用图传播常量参数与返回值。

五、循环与全局优化

在 IR 结构稳定后，优化重点转向循环区域与全局代码布局，旨在降低高频执行路径上的开销。

5.1 循环变换

`LoopUnrollingPass` 针对可静态确定迭代次数的循环进行展开。它通过模拟执行判定循环的 Trip Count，若循环体规模与迭代次数的乘积在阈值内，则进行完全展开。

```
// 循环展开核心：克隆与重连
for (int i = 0; i < tripCount; i++) {
    // 克隆循环体
    Map<Value, Value> map = cloneLoopBody(loop);
    // 替换归纳变量: IV → Init + i * Step
    Value currentIV = new ConstInt(init + i * step);
    map.get(iv).replaceAllUsesWith(currentIV);

    // 连接控制流: 上一轮 Latch → 当前 Header
    if (i > 0) {
        lastLatch.getTerminator().replaceTarget(loop.getHeader(),
        map.get(loop.getHeader()));
    }
    lastLatch = map.get(loop.getLatch());
}
// 处理出口: 最后一次 Latch → Exit
lastLatch.getTerminator().replaceTarget(loop.getHeader(), loop.getExit());
```

代码 7 循环展开核心逻辑

随后，`TailRecursionEliminationPass` 将满足特定约束（如无副作用、尾调用形式）的尾递归转化为循环。通过引入新的 Header 块与 Phi 节点，将递归调用转化为跳转，消除了函数调用的栈帧开销。

```
// 将尾调用转化为跳转
for (CallInst call : tailCalls) {
    // 更新参数 Phi 的入边: 使用调用实参
    for (int i = 0; i < args.size(); i++) {
        argPhis.get(i).addIncoming(call.getOperand(i + 1), call.getParent());
    }
    // 替换 Call 为跳转到新的循环头
    new BrInst(treHeader, call.getParent());
    call.remove();
}
```

代码 8 尾递归消除逻辑

5.2 全局值编号

GvnPass 利用支配树进行公共子表达式消除 (CSE)。它在遍历支配树的过程中维护一个值表, 将计算结果映射到规范化的键值 (GvnKey)。若后续指令计算出相同的键值, 则直接复用前序结果。

```
// 在支配树遍历中查表去重
private void runOnBlock(BasicBlock bb) {
    for (Instruction inst : bb.getInstructions()) {
        GvnKey key = new GvnKey(inst, this);
        if (valueTable.containsKey(key)) {
            // 发现冗余计算: 用已有值替换当前指令
            inst.replaceAllUsesWith(valueTable.get(key));
            inst.remove();
        } else {
            // 记录新值
            valueTable.put(key, inst);
        }
    }
    // ... 递归处理子节点, 并在回溯时恢复表状态 ...
}
```

代码 9 GVN 查表去重逻辑

5.3 代码移动

代码移动结合了 LICM (Loop Invariant Code Motion) 与 GCM (Global Code Motion)。LICM 识别循环不变式并将其提升至 Preheader; GCM 则根据数据依赖关系, 将指令调度到满足支配关系且循环深度最小的位置。

```
// 确定最晚放置点: 选择循环深度最小的 LCA
private BasicBlock scheduleLate(Instruction inst) {
    BasicBlock lca = null;
    for (User user : inst.getUsers()) {
        BasicBlock useBlock = getUserBlock(user, inst);
        lca = findLca(lca, useBlock);
    }

    // 在 LCA 到 EarlyBlock 的路径上, 选择循环深度最小的块
    BasicBlock best = lca;
    while (lca != earlyBlock) {
        lca = idom.get(lca);
        if (loopDepth(lca) < loopDepth(best)) {
            best = lca;
        }
    }
    return best;
}
```

代码 10 GCM 最晚放置点选择

5.4 强度削减

`LoopStrengthReductionPass` 将循环内的乘法与地址计算转化为增量更新。它识别循环归纳变量，并为形如 `i * c` 或 `GEP(base, i)` 的计算引入新的 Phi 节点，在循环体中通过加法更新该 Phi，从而移除高代价的乘法与重复地址计算。

```
// 强度削减: GEP(base, i) → PtrPhi; PtrPhi += step
PhiInst ptrPhi = new PhiInst(gepType, "lsr.ptr", loop.getHeader());
ptrPhi.addIncoming(initialGep, preheader); // Init: GEP(base, init)

// 在 Latch 处插入增量更新
GetElementPtrInst nextPtr = new GetElementPtrInst(ptrPhi, step);
ptrPhi.addIncoming(nextPtr, loop.getLatch());

// 替换原 GEP 使用
originalGep.replaceAllUsesWith(ptrPhi);
```

代码 11 强度削减逻辑

六、后端代码生成

后端负责将优化后的 IR 映射为 MIPS 指令，重点解决 Phi 消除、寄存器分配与指令级优化。

6.1 Phi 消除

在代码生成前，必须将 SSA 形式的 Phi 指令转化为实际的数据传送。`fillPhis` 方法在控制流边上插入并行拷贝序列。为了处理并行拷贝中的依赖循环（如 swap 操作），算法在必要时引入临时栈槽打破循环。

```
// 处理并行拷贝中的环：利用栈作为临时空间
private void handleCycles(Map<Phi, Value> assignments) {
    // 1. 将环中所有源值压栈保存
    for (Value src : assignments.values()) {
        pushToStack(src);
    }
    // 2. 从栈中弹出并写回目的寄存器
    // 注意：需按保存顺序的逆序或对应关系恢复
    for (Phi dst : assignments.keySet()) {
        Value val = popFromStack();
        move(dst, val);
    }
}
```

代码 12 并行拷贝环处理

6.2 寄存器分配

寄存器分配采用图着色算法。LivenessAnalysis 计算活跃区间，构建干涉图。

GraphColoringRegAlloc 根据干涉图进行着色。在无法着色时，根据溢出代价 (Spill Cost) 选择变量溢出到栈上。分配过程中会优先利用 move 指令的提示 (Coalescing) 来减少寄存器间的拷贝。

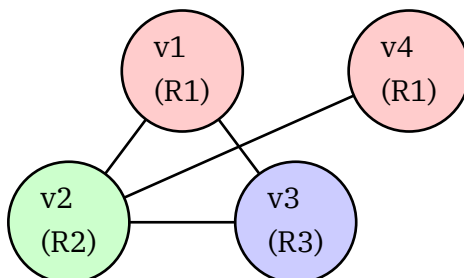


图 3 寄存器分配干涉图着色示例

```

// 特殊处理 Phi 的活跃性: Phi 的使用发生在“前驱边”上
private void computeDefUse() {
    for (Instruction inst : bb.getInstructions()) {
        if (inst instanceof PhiInst phi) {
            // Phi 的操作数不计入当前块的 Use
            // 而是计入前驱块的 PhiUse 集合
            for (int i = 0; i < phi.getNumOperands(); i++) {
                BasicBlock pred = phi.getIncomingBlock(i);
                phiUse.get(pred).add(phi.getIncomingValue(i));
            }
        } else {
            // 普通指令: 操作数计入 Use, 结果计入 Def
            // ...
        }
    }
}

```

代码 13 Phi 指令活跃性分析

6.3 除法优化

针对高权重的除法指令，后端使用魔数法 (Magic Number) 将除以常数转化为乘法与移位操作。`chooseMultiplier` 算法计算对应的乘数与移位量，确保结果与有符号除法一致。

```
// 应用魔数优化生成指令序列:  $n / d \rightarrow (n * m) \gg \text{shift}$ 
public void visitSdiv(SdivInst inst) {
    if (isConstant(inst.rhs)) {
        int d = inst.rhs.getInt();
        MultiplierInfo info = chooseMultiplier(d);

        // 生成 MIPS 指令序列
        // 1. 高位乘法: mult $n, $m; mfhi $tmp
        emit("mult", reg(n), li(info.multiplier));
        emit("mfhi", tmp);

        // 2. 移位与符号修正
        if (info.shift > 0) emit("sra", tmp, tmp, info.shift);
        emit("srl", sign, reg(n), 31);
        emit("addu", res, tmp, sign);

        return;
    }
    // ... 处理非常量除法 ...
}
```

代码 14 除法魔数优化代码生成

七、总结

本项目构建了一个以 `FinalCycle` 为约束的优化流水线。中端通过 SSA 形式化解了内存与控制流冗余，后端通过寄存器分配与指令级优化落实了硬件层面的开销削减。各阶段优化紧密配合，最终在保证语义正确的前提下实现了性能指标的提升。